IS115 Project: Last-Mile Delivery Planning

Objective

In this project, your team helps a logistics company develop a dispatch algorithm to deliver parcels to a town. The objective is to improve a performance metric of your choice, such as operating cost, on-time delivery rate or average delivery time.

You are given operational data on parcels and vehicles, distances between locations, a simulator and a baseline dispatch algorithm. First, pick an operational performance metric to improve. Start by establishing the baseline performance. Then propose several dispatch algorithms, analyse their time and space complexity, assess their advantages and limitations, and recommend a final approach.

Your solution must meaningfully incorporate at least one algorithm and one data structure taught in class.

Operating Model

This section describes how the last-mile delivery operation works at a high level. The project package given to you will include a simulator that models one day of operations, including parcel arrivals, dispatch planning and deliveries.

Hub and parcels

One hub serves as the depot, with parcels arriving throughout the day until 17:30. Each parcel has a unique identifier, an arrival time at the hub, a delivery location, a size and a preferred delivery window specified by start and end times. The delivery window is a soft constraint, so early and late deliveries are allowed. Some parcels may remain undelivered at the end of the day, as achieving 100% delivery is not a hard constraint.

Vehicles and trips

A fixed fleet of ten homogeneous vehicles loads parcels at the hub, delivers the parcels to one or more delivery locations and returns to the hub. All vehicles start at the hub, fully charged and available at 08:00. Vehicles travel at 60 km/h and can make multiple trips, receiving a full recharge each time they return.

Planning and delivery

Planning takes place every ten minutes from 08:00 through 17:30. Each planning round considers available vehicles in turn, starting with the vehicle that has been idle the longest. Once a vehicle's dispatch is confirmed, it departs 30 minutes later. This delay simulates parcel loading and vehicle charging. No additional charging delay is applied. Delivery service time is zero.

Hard constraints

For each trip, the total parcel size must not exceed the vehicle capacity of 30, and the distance travelled must not exceed the vehicle range of 320 km. Every vehicle must return to the hub by 18:00.

Project Package

The project package has the following folders. All paths and commands below are relative to the project root. Python 3.9 or later is required.

- `simulator/`: The supplied framework models parcel arrivals, dispatch planning, vehicle deliveries and vehicle returns to the hub. Do not modify these files.

File	Contents
`simulator.py`	Runs the simulation, schedules events, validates dispatch decisions and writes the event log.
`loaders.py`	Reads and validates the parcel, distance and vehicle configuration files.
`models.py`	Defines the simulation events and the structures of parcel, configuration and dispatch context records.

- `data/`: Contains twenty days of historical parcel data in `data/demand/day_001.csv` through `data/demand/day_020.csv`. All days share `data/dist.csv` and `data/config.json`. Times in the parcel files use `HH:MM` format, while the simulator and event log use minutes after midnight. Do not modify these files.

File	Contents
`data/demand/day_XXX.csv`	One day's parcel IDs, arrival times at the hub, delivery locations, sizes and preferred delivery windows (start and end times)
`data/dist.csv`	Directed distances between locations
`data/config.json`	Number of vehicles, vehicle capacity and range

- `solution/`: Contains your dispatch policies, performance monitor and any helper modules. The supplied FIFO policy is in `solution/policies/baseline.py`; leave it unchanged. Create a separate module in `solution/policies/` for each approach, and implement your metric in the supplied `solution/performance_monitor.py` template. Keep the `__init__.py` package files in place.
- `results/`: Generated experiment outputs, organised by policy and day, for example `results/baseline/day_001/log.txt`. This folder is created when you run the simulator. Use a separate output folder for each policy and day to preserve previous results.

See Appendix A: Folder Structure for a summary of the project folders, files and their roles.

Tasks

1. Define your performance metric

Choose a performance metric and rationalize its significance for last-mile deliveries. Define how you calculate it, including any assumptions. Any metric that leads to a trivial approach will receive zero marks.

2. Establish a baseline performance

Run the simulator using the supplied `baseline.py` policy to generate an event log. Then write your metric calculation in `solution/performance_monitor.py` to analyse the log and measure baseline performance. Follow these steps from the project root.

Step 1: Run the baseline simulation.

```
python3 -m simulator --policy solution.policies.baseline \
    --data data --day day_001 \
    --output results/baseline/day_001
```

The `--data data` option loads the scenario inputs from `data/`, and `--day day_001` selects the day's parcel data. This runs `solution/policies/baseline.py` and saves the event log to `results/baseline/day_001/log.txt`.

Step 2: Inspect the log.

Each line is a JSON event record. For example, two departure records could be:

```
{"event_type": "hub_departure", "time": 510.0, "vehicle_id": 1, "trip_id": 1}
{"event_type": "hub_departure", "time": 510.0, "vehicle_id": 2, "trip_id": 1}
```

Times are minutes after midnight, so `510.0` means 08:30. Each `hub_departure` records one trip; different vehicles can have the same `trip_id`.

For more information, refer to Appendix B on reading log.txt.

Step 3: Write your performance monitor.

Implement your chosen metric in `calculate_metrics` in `solution/performance_monitor.py`. The supplied script reads the log into `events` and also passes the selected day's parcel records, distances and vehicle configuration to this function. See Appendix C for the inputs and required return value.

Here is a function that illustrates total trip count as the performance metric:

```
def calculate_metrics(parcels, distances, config, events):
    number_of_trips = 0
    for event in events:
        if event["event_type"] == "hub_departure":
            number_of_trips += 1
    return {"number_of_trips": number_of_trips}
```

Step 4: Run the performance monitor and check the results.

Run your performance monitor on the baseline log:

```
python3 -m solution.performance_monitor \
    --data data --day day_001 \
    --log results/baseline/day_001/log.txt
```

The performance monitor saves `metrics.json` beside the log. Open `results/baseline/day_001/metrics.json` to check your calculated metric values. For a log containing exactly the two departures illustrated above, the trip-count example writes:

```
{
  "number_of_trips": 2
}
```

Repeat for `day_002` through `day_020`, using a separate results folder for each day. Keep these baseline results for comparison with your policies.

3. Develop your dispatch algorithms

Write each dispatch policy in a separate file under `solution/policies/`, such as `heuristic_best.py`. Put shared helper functions in `solution/policies/helpers.py` and import them with `from solution.policies.helpers import ...`. Use only the Python standard library. Leave `baseline.py` and the simulator unchanged.

Each policy module must expose the following function, keeping its interface unchanged. See Appendix C for a concise field reference:

```
def dispatch(parcel, context):
    # Return parcel IDs in delivery order
    return []
```

The simulator calls this function once for each available vehicle during a planning round. Both arguments are Python dictionaries. Each call must return an ordered list of parcel IDs for the given available vehicle.

`parcel` maps parcel IDs to records for all parcels that have arrived, including unassigned, assigned, and delivered. For example, at 09:00:

```
parcel = {
    "P001": {
        "arrival_time": 480.0,           # 08:00
        "delivery_location_id": "L001",
        "delivery_window_start": 540.0,  # 09:00
        "delivery_window_end": 660.0,    # 11:00
        "size": 4.0,
        "delivery_time": None,
        "vehicle_assigned": None,
        "trip_id": None,
    },
    "P002": {
        "arrival_time": 510.0,           # 08:30
        "delivery_location_id": "L002",
        "delivery_window_start": 540.0,  # 09:00
        "delivery_window_end": 720.0,    # 12:00
        "size": 6.0,
        "delivery_time": None,
        "vehicle_assigned": None,
        "trip_id": None,
    },
}
```

A parcel is available for selection when both `vehicle_assigned` and `delivery_time` are `None`.

`context` provides the current planning time, the vehicle being considered, capacity, distance lookup, departure delay, return cutoff and fleet availability. The following example uses a smaller distance matrix and fleet for illustration:

```
context = {
    "current_time": 540.0,          # 09:00
    "vehicle_id": 1,               # Plan for this vehicle
    "capacity": 30.0,
    "travel_distance": {           # Directed distances in kilometres
        "HUB":  {"HUB": 0.0, "L001": 10.0, "L002": 15.0},
        "L001": {"HUB": 10.0, "L001": 0.0, "L002": 8.0},
        "L002": {"HUB": 15.0, "L001": 8.0, "L002": 0.0},
    },
    "departure_delay": 30.0,       # Depart 30 minutes after planning
    "return_cutoff": 1080.0,       # Return by 18:00
    "max_trip_distance_km": 320.0,
    "fleet": {
        1: {"available_at": 540.0, "max_trip_distance_km": 320.0},
        2: {"available_at": 600.0, "max_trip_distance_km": 320.0},
    },
}
```

All times are in minutes after midnight. The distance lookup contains distances in kilometres; at 60 km/h, each distance also equals the travel time in minutes. `fleet` shows each vehicle's planned availability, but the current call makes decisions only for `context["vehicle_id"]`.

Return an ordered list of parcel IDs for the vehicle's trip. For the example above, returning `["P001", "P002"]` assigns vehicle 1 the route **HUB** → **L001** → **L002** → **HUB**, departing at 09:30. Returning `[]` leaves the vehicle available for the next planning round. The simulator validates each route before assigning parcels and raises an error for an invalid route.

Treat both input dictionaries as read-only snapshots. Do not modify the simulator's state.

Your dispatch algorithm may access only the information supplied through `parcel` and `context`. It must not read files in `data/` during a simulation.

Select a policy with `--policy`, using its Python module path without the `.py` extension. For example, after creating `solution/policies/heuristic_best.py`, run:

```
python3 -m simulator --policy solution.policies.heuristic_best \
    --data data --day day_001 \
    --output results/heuristic_best/day_001
```

The event log is saved to `results/heuristic_best/day_001/log.txt`. Run your performance monitor on this log.

4. Experiment on multiple approaches

Develop several dispatch policies and compare them with the baseline using the same historical data. Perform extensive experiments and analyze their performances on your chosen metric. Analyze time and space complexity. Discuss potential trade-offs. Explain where each approach performs well or poorly. Use the results to arrive at a recommended approach.

Deliverables

Submit your code, results, README and report together in one ZIP file to eLearn.

- Your `solution/` folder, with all the policies you explored and helper functions in `solution/policies/` and your metric implemented in `performance_monitor.py`. Keep `policies/baseline.py` and all `__init__.py` files unchanged.
- Your `results/` folder containing `log.txt` and `metrics.json` for each policy you evaluated, organised by policy and day as shown in Appendix A.
- A `README.md` at the project root with commands to reproduce the baseline and policy results, including how to run the performance monitor.
- A report of your findings of no more than 8 pages, excluding references. The report should cover problem statement and motivation, algorithmic approaches, computational complexity, experimental results, conclusion and references.

Deadline

- **Friday, Week 13 (13 Nov 2026), 23:59**.

Team Size

- 2–3 members per team.

Rubric

The submitted work is assessed as follows:

- Problem statement and motivation: 10%.
- Algorithmic approaches: 20%.
- Computational complexity analysis: 20%.
- Experimental results: 20%.
- Conclusion: 10%.
- Code reproducibility and references: 10%.

10% of your project will come from a question in the final exam.

On Use of AI

You may use only the provided **Gemini Gem** (link) as your coding assistant, and only to generate boilerplate code. You must formulate the algorithmic approaches yourselves, but you can ask AI to explain and clarify concepts learnt in the course. You are not allowed to one-shot a solution. Include all prompts and AI responses in the references section of your report.

AI can make mistakes. Please verify its output.

Project Enquiries

For enquiries about the project, please write directly to Pang-Jin Tan at pjtan@smu.edu.sg.

Office hours are Wednesdays (14:00–16:00) and Fridays (10:00–12:00) during Weeks 11–13, online at `https://smu.sg/pjtan`. Appointments are required.

Appendix A: Folder Structure

```
project/
├── simulator/                  # Do not modify
│   ├── simulator.py
│   ├── loaders.py
│   ├── models.py
│   ├── __main__.py
│   └── __init__.py
├── data/                       # Do not modify
│   ├── demand/
│   │   ├── day_001.csv
│   │   ├── ...
│   │   └── day_020.csv
│   ├── dist.csv
│   └── config.json
├── solution/
│   ├── __init__.py             # Do not modify
│   ├── performance_monitor.py  # TODO: Implement your performance metric
│   └── policies/
│       ├── __init__.py         # Do not modify
│       ├── baseline.py         # Baseline policy; do not modify
│       ├── helpers.py          # TODO: Write your helper functions here
│       └── heuristic_best.py   # TODO: Create your policy here
├── results/
│   ├── baseline/               # Results from baseline policy
│   │   ├── day_001/
│   │   │   ├── log.txt         # Generated by simulator.py
│   │   │   └── metrics.json    # Generated by performance_monitor.py
│   │   ├── ...
│   │   └── day_020/
│   │       ├── log.txt
│   │       └── metrics.json
│   ├── heuristic_best/         # Results from your policy
│   │   ├── day_001/
│   │   │   ├── log.txt         # Generated by simulator.py
│   │   │   └── metrics.json    # Generated by performance_monitor.py
│   │   ├── ...
│   │   └── day_020/
│   │       ├── log.txt
│   │       └── metrics.json
│   └── ...
└── README.md
```

Appendix B: Reading log.txt

Format

Each simulation writes `log.txt` in its selected results folder, for example `results/baseline/day_001/log.txt`. The file uses JSON Lines: each non-empty line is a separate JSON object. There is no enclosing array or comma between records. Read it one line at a time with `json.loads`, rather than using `json.load` on the whole file.

Records appear in event-processing order, with times in minutes after midnight. Times may have fractional minutes; several records may share a time. Each record has `event_type` and `time`, plus the fields listed below.

Event types

`event_type`	Additional fields	Meaning
`parcel_arrival`	`parcel_id`	A parcel arrives at the hub and becomes available for assignment.
`vehicle_return`	`vehicle_id`	A vehicle is available at the hub with full range. Also recorded at 08:00 to initialise each vehicle; these initial records are not completed trips.
`dispatch_planning`	None	A planning event calls the selected policy's `dispatch` function once for each available vehicle.
`hub_departure`	`vehicle_id`, `trip_id`	A vehicle leaves the hub for one trip.
`delivery`	`vehicle_id`, `trip_id`, `parcel_id`	A parcel is delivered. This record alone does not indicate whether delivery was within its preferred window.

Field meanings

Field	Type	Meaning
`event_type`	String	The event name from the table above.
`time`	Number	Event time in minutes after midnight: `480.0` is 08:00 and `510.5` is 08:30:30.
`parcel_id`	String	Parcel identifier matching the selected day's demand CSV.
`vehicle_id`	Integer	Vehicle identifier, starting at 1.
`trip_id`	Integer	Trip number for that vehicle during the simulated day, starting at 1 and increasing by 1 for each trip. Different vehicles may have the same trip number.

Fields that do not apply to an event are omitted. In particular, `vehicle_return` has no `trip_id`. Parcel destinations, sizes and delivery windows are in `data/demand/day_XXX.csv`, rather than repeated in the log. Use `parcel_id` to match delivery records to those inputs when calculating your metric.

Reading the file in Python

```
import json

with open("results/baseline/day_001/log.txt", encoding="utf-8") as stream:
    events = [json.loads(line) for line in stream if line.strip()]
```

This produces a list of event dictionaries. The supplied `performance_monitor.py` already reads the log this way and passes `events` to `calculate_metrics`. Only use a log from a successfully completed simulation: if a run fails, its log may be incomplete.

Appendix C: Function Interfaces

dispatch(parcel, context)

Called once for each available vehicle during a planning round. Both arguments are dictionary snapshots. All times are numbers in minutes after midnight (e.g. 540 is 09:00); distances are in kilometres.

Return: a Python list of unique, unassigned parcel ID strings in delivery order for the current vehicle, e.g. `["P001", "P002"]`. Return `[]` to skip this vehicle for this round. The trip starts and ends at `HUB`; do not include `HUB` in the list. The route must satisfy capacity, range and return-cutoff constraints.

parcel: a dictionary keyed by parcel ID, containing all parcels that have arrived, including assigned and delivered parcels. Each record has:

Field	Meaning
`arrival_time`	Time the parcel arrived at the hub.
`delivery_location_id`	Destination ID used in the distance lookup.
`delivery_window_start`	Start of the preferred delivery window.
`delivery_window_end`	End of the preferred delivery window.
`size`	Size of parcel in the same units as vehicle capacity.
`delivery_time`	Actual delivery time; `None` until delivered.
`vehicle_assigned`	Assigned vehicle ID; `None` if unassigned.
`trip_id`	Trip number within that vehicle; `None` until assigned.

context: a dictionary describing the current decision:

Field	Meaning
`current_time`	Time of this planning round.
`vehicle_id`	ID of the available vehicle whose trip you are planning.
`capacity`	Total capacity a vehicle can accommodate.
`travel_distance`	Nested dictionary: `[origin][destination]` gives the directed distance from origin to destination, including routes to and from `HUB`.
`departure_delay`	Minutes before departure: departure time is `current_time + departure_delay`.
`return_cutoff`	Latest permitted return time at the hub.
`max_trip_distance_km`	Maximum distance for the whole trip, including return to the hub.
`fleet`	Dictionary keyed by vehicle ID. Each record has `available_at` (time ready at the hub) and `max_trip_distance_km` (full per-trip range). An idle vehicle may have an availability time earlier than the current time.

calculate_metrics(parcels, distances, config, events)

Called once by the performance monitor after it loads the selected day's input files and one simulation run's log. It receives the complete day's inputs, including parcels that were never delivered.

Return: a dictionary with string metric names and JSON-serializable values, e.g. `{"score": 0.8}`. Use finite numbers (no NaN or infinity). The supplied script writes this dictionary to `metrics.json`; the function need not write a file.

parcels: a list of row dictionaries from the day's demand CSV. All values are strings; convert numeric values before arithmetic. Unlike `dispatch()`, this argument has a `parcel_id` field in each row and has no assignment or delivery-state fields.

Field	Meaning
`parcel_id`	Parcel identifier used to match event records.
`arrival_time`	Hub arrival time as an `HH:MM` string.
`delivery_location_id`	Destination ID used in the distance rows.
`delivery_window_start`	Preferred window start as an `HH:MM` string.
`delivery_window_end`	Preferred window end as an `HH:MM` string.
`size`	Capacity units as a string; convert with `float()`.

distances: a list of row dictionaries from `dist.csv`, rather than a nested lookup. Each row contains `location_1` (origin ID), `location_2` (destination ID), and `dist` (directed distance in kilometres as a string; convert with `float()`).

config: a dictionary from `config.json`, with numeric values:

Field	Meaning
`number_of_vehicles`	Number of vehicles in the fleet.
`capacity`	Maximum parcel capacity per vehicle, in the same units as size.
`vehicle_range_km`	Maximum distance per trip, including return to the hub.

events: a list of events in chronological order. Each has `event_type` and numeric `time` in minutes after midnight, plus event-specific IDs. See Appendix B for all event types and fields. Match deliveries to `parcels` by `parcel_id` to evaluate delivery windows or identify undelivered parcels.